

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САУ

ОТЧЕТ
по Индивидуальному домашнему заданию
по дисциплине «Программируемые логические контроллеры и
промышленные сети»
Тема: СОЗДАНИЕ ПОЛЬЗОВАТЕЛЬСКИХ БИБЛИОТЕК

Студент гр. 2491

Недовизий А.И.

Преподаватель

Богданова С.М.

Санкт-Петербург

2025

Цель работы: создать программу для управления системой освещения в среде CoDeSys на примере написания простой программы.

Задание для 1го варианта:

Создать и применить библиотеку, хранящую в себе следующие элементы: функциональный блок для генерации короткого импульса по фронту и срезу входного сигнала, функциональный блок для формирования и "защелкивания" командной посылки, функциональный блок для реализации защитной логики по нескольким сигналам.

Пояснение:

- Первый ФБ должен генерировать короткий импульс при смене состояния входного сигнала;
- Второй ФБ формирует и фиксирует ("защелкивает") команду только в том случае, если в момент ее поступления было дано разрешение. Команда остается активной до явного сброса.
- Третий ФБ обрабатывает несколько аварийных сигналов, определяя, какой из них сработал первым, и формирует общий аварийный сигнал с кодом неисправности.

Как применить в программе:

Создайте тестовую программу, в которой продемонстрируйте взаимодействие всех трех библиотечных элементов: подайте сигнал с кнопки на вход первого ФБ, используйте его выходной импульс по фронту как триггер для второго ФБ, при этом разрешение на выполнение команды должно формироваться по отсутствию аварийных сигналов с выхода третьего ФБ. Настройте три аварийных сигнала и убедитесь, что при их активации команда сбрасывается, а код первой аварии корректно отображается. Проверьте, что команда надежно защелкивается только при одновременном наличии разрешения и триггерного импульса, и что каждый импульс от кнопки длится ровно один цикл программы независимо от длительности удержания кнопки.

Ход работы

1) Реализация программы на языке ST.

Создадим функциональный блок PulseEdge, который будет генерировать короткий импульс при смене состояния входного сигнала.

```
1  FUNCTION_BLOCK PulseEdge
2  VAR_INPUT
3      InputSignal: BOOL;           // Входной сигнал для мониторинга
4  END_VAR
5
6  VAR_OUTPUT
7      Pulse: BOOL;                // Импульс при любом изменении состояния
8  END_VAR
9
10 VAR
11     PreviousSignalState: BOOL;    // Хранение предыдущего состояния
12 END_VAR
13
14 // Генерация импульса при изменении состояния
15 Pulse := FALSE; // Сброс выхода
16
17 IF InputSignal <> PreviousSignalState THEN
18     Pulse := TRUE; // Импульс длительностью один цикл
19 END_IF;
20
21 // Сохранение текущего состояния для следующего цикла
22 PreviousSignalState := InputSignal;
```

Рисунок 1 – Реализация программы на языке ST

2) Реализация программы на языке ST.

Создадим функциональный блок CommandLatch, который формирует и фиксирует (“защелкивает”) команду только в том случае, если в момент ее поступления было дано разрешение. Команда остается активной до явного сброса

```

FUNCTION_BLOCK CommandLatch
VAR_INPUT
    Cmd: BOOL;           // Командный импульс
    Enable: BOOL;        // Разрешение выполнения
    Reset: BOOL;         // Сброс команды
END_VAR

VAR_OUTPUT
    LatchedCmd: BOOL;    // Защелкнутая команда
END_VAR

VAR
    CommandState: BOOL := FALSE; // Внутреннее состояние
END_VAR

IF Reset THEN
    CommandState := FALSE;        // Сброс команды
ELSIF Cmd AND Enable THEN
    CommandState := TRUE;         // Защелкивание команды
END_IF;

LatchedCmd := CommandState;      // Выходное значение

```

Рисунок 2 – Реализация программы на языке ST

3) Реализация программы на языке ST.

Создадим функциональный блок Protection, который обрабатывает несколько аварийных сигналов, определяя, какой из них сработал первым, и формирует общий аварийный сигнал с кодом неисправности.

```

1  FUNCTION_BLOCK Protection
2  VAR_INPUT
3      Alarm1: BOOL;      // Аварийный сигнал 1
4      Alarm2: BOOL;      // Аварийный сигнал 2
5      Alarm3: BOOL;      // Аварийный сигнал 3
6      Reset: BOOL;       // Сброс аварий
7  END_VAR
8
9  VAR_OUTPUT
10     GeneralAlarm: BOOL; // Общий аварийный сигнал
11     FaultCode: INT;     // Код активной аварии
12 END_VAR
13
14 VAR
15     FirstAlarm: INT := 0;      // Код первой по времени аварии
16     AlarmActive: BOOL := FALSE; // Флаг активности аварии
17     Alarm1_Prev: BOOL := FALSE; // Предыдущее состояние Alarm1
18     Alarm2_Prev: BOOL := FALSE; // Предыдущее состояние Alarm2
19     Alarm3_Prev: BOOL := FALSE; // Предыдущее состояние Alarm3
20 END_VAR

```

```

7
8     // Если Alarm2 только что активировался (и еще не было аварии)
9     IF Alarm2 AND NOT Alarm2_Prev AND NOT AlarmActive THEN
10         FirstAlarm := 2;
11         AlarmActive := TRUE;
12     END_IF
13
14     // Если Alarm3 только что активировался (и еще не было аварии)
15     IF Alarm3 AND NOT Alarm3_Prev AND NOT AlarmActive THEN
16         FirstAlarm := 3;
17         AlarmActive := TRUE;
18     END_IF
19 END_IF;
20
21 // Сохраняем текущие состояния для следующего цикла
22 Alarm1_Prev := Alarm1;
23 Alarm2_Prev := Alarm2;
24 Alarm3_Prev := Alarm3;
25
26 // Формирование выходов
27 GeneralAlarm := AlarmActive;
28 FaultCode := FirstAlarm;
29
30 // Сброс аварий (только когда все аварии сняты и есть команда сброса)
31 IF Reset AND NOT (Alarm1 OR Alarm2 OR Alarm3) THEN
32     AlarmActive := FALSE;
33     FirstAlarm := 0;
34     Alarm1_Prev := FALSE; // Сброс памяти предыдущих состояний
35     Alarm2_Prev := FALSE;
36     Alarm3_Prev := FALSE;
37 END_IF;

```

Рисунок 3 – Реализация программы на языке ST

4) Реализация программы на языке ST.

Создадим тестовую программу, в которой будет взаимодействие всех трех библиотечных элементов: подадим сигнал с кнопки на вход первого ФБ, используем его выходной импульс по фронту как триггер для второго ФБ. Настроим три аварийных сигнала и убедимся, что при их активации команда сбрасывается, а код первой аварии корректно отображается.

```
PROGRAM TestProgram
VAR_INPUT
    Start_Button: BOOL;    // Кнопка команды запуска
    Emergency_1: BOOL;     // Аварийный сигнал 1
    Emergency_2: BOOL;     // Аварийный сигнал 2
    Emergency_3: BOOL;     // Аварийный сигнал 3
    Cmd_Reset: BOOL;       // Сброс команды
    Emerg_Reset: BOOL;     // Сброс аварий
    Enable_Command: BOOL;  // разрешение на команду
END_VAR

VAR_OUTPUT
    Active_Command: BOOL;  // Активированная команда
    Error_Code: INT;       // Код аварии
END_VAR

VAR
END_VAR

VAR
    // Экземпляры функциональных блоков
    PulseGen: PulseEdge;
    CmdLatch: CommandLatch;
    Protect: Protection;
END_VAR

PulseGen(InputSignal := Start_Button);

// FB Protection
Protect(
    Alarm1 := Emergency_1,
    Alarm2 := Emergency_2,
    Alarm3 := Emergency_3,
    Reset := Emerg_Reset,
    GeneralAlarm => ,           // Общая авария
    FaultCode => Error_Code    // Код ПЕРВОЙ ПО ВРЕМЕНИ тревоги
);
// FB CommandLatch
CmdLatch(
    Cmd := PulseGen.Pulse,      // Импульс как триггер
    Enable := Enable_Command AND (NOT Protect.GeneralAlarm),
    Reset := Cmd_Reset OR Protect.GeneralAlarm, // Сброс по кнопке ИЛИ при аварии
    LatchedCmd => Active_Command // Защелкнутая команда
);
```

Рисунок 4 – Реализация программы на языке ST

Покажем пример работы программы на рисунке 5.

Выражение	Тип	Значение	Подготовленное ...	Адрес	Комментарий
Start_Button	BOOL	FALSE			Кнопка команды запуска
Emergency_1	BOOL	FALSE			Аварийный сигнал 1
Emergency_2	BOOL	FALSE			Аварийный сигнал 2
Emergency_3	BOOL	FALSE			Аварийный сигнал 3
Cmd_Reset	BOOL	FALSE			Сброс команды
Emerg_Reset	BOOL	FALSE			Сброс аварий
Enable_Command	BOOL	TRUE			разрешение на команду
Active_Command	BOOL	TRUE			Активированная команда
Error_Code	INT	0			Код аварии
PulseGen	PulseEdge				Экземпляры функциональных блоков
InputSignal	BOOL	FALSE			Входной сигнал для мониторинга
Pulse	BOOL	FALSE			Импульс при любом изменении состояния
PreviousSignalState	BOOL	FALSE			Хранение предыдущего состояния
CmdLatch	CommandLatch				
Cmd	BOOL	FALSE			Командный импульс
Enable	BOOL	TRUE			Разрешение выполнения
Reset	BOOL	FALSE			Сброс команды
LatchedCmd	BOOL	TRUE			Защелкнутая команда
CommandState	BOOL	TRUE			Внутреннее состояние
Protect	Protection				
Alarm1	BOOL	FALSE			Аварийный сигнал 1
Alarm2	BOOL	FALSE			Аварийный сигнал 2
Alarm3	BOOL	FALSE			Аварийный сигнал 3
Reset	BOOL	FALSE			Сброс аварий
GeneralAlarm	BOOL	FALSE			Общий аварийный сигнал
FaultCode	INT	0			Код активной аварии
FirstAlarm	INT	0			Код первой по времени аварии
AlarmActive	BOOL	FALSE			Флаг активности аварии
Alarm1_Prev	BOOL	FALSE			Предыдущее состояние Alarm1
Alarm2_Prev	BOOL	FALSE			Предыдущее состояние Alarm2
Alarm3_Prev	BOOL	FALSE			Предыдущее состояние Alarm3

Рисунок 5 – Пример работы программы при подаче импульса и срабатывании защелкивания...

Вывод: Выполняя данную лабораторную работу, мы продемонстрировали практическую ценность модульного подхода в программировании ПЛК. Созданная библиотека из трех функциональных блоков (генератор импульсов, команда с защелкой и защитная логика) успешно решает типовые задачи промышленной автоматизации. Интеграция блоков в тестовой программе показала их корректное взаимодействие: импульс от кнопки через генератор поступает в командный блок только при отсутствии аварий, что обеспечивает безопасное управление.